

Recovery Risk Model Audit

One model helps decide which borrower an agent visits tomorrow. This is what it is, where it sits, what it looks at, and how well it has actually been shown to work.

MODEL	recovery_risk
VERSION	1.1.0
LIVE SINCE	2026-09-08
GATES	12 / 12 PASS
AUDITED	2026-09-11

WHAT IT DOES

It answers one question about one borrower: are they about to pay something, or not?

Every night the system looks at each overdue account and asks the model: **over the next 30 days, will this borrower make a payment big enough to matter?** "Big enough" has a fixed definition — at least 80% of whichever is smaller, the arrears or one instalment. The model answers with a percentage.

That percentage is not a verdict on the person. It is a ranking tool. With 931 overdue cases and roughly 214 visits available in a day, somebody has to choose which ones get visited. The model's job is to make that choice better than a list sorted by amount owed.

One point of confusion worth clearing up. Internally the model predicts the *bad* outcome — the chance of *no* payment — so a higher raw number is worse. Everything shown to a manager is flipped into the friendly direction: chance of recovery. On last night's run that averaged **14.9%** across allocated cases. A 14% figure on screen is the model saying "roughly one borrower like this in seven pays next month", which on a book this far overdue is the honest answer rather than a pessimistic one.

$$\text{expected recovery} = (\text{target} - \text{already collected}) \times \text{chance of recovery}$$

This is the only place the model's number becomes money. The rupee figure on the allocation screen is that multiplication and nothing else — which is why an over-confident model would misprice every single case by however much it is wrong by.

WHERE IT SITS

Inside the nightly run, between the bank file arriving and tomorrow's routes being drawn.

Nothing about the model is interactive. It runs once a night, unattended, in a fixed order. The step marked in teal is the only one where the model makes a prediction; the 19:15 step is where last month's predictions get marked right or wrong.

19:15

Yesterday's homework gets marked

Predictions that have now had their full 30 days are compared against what the borrower actually did, and a drift monitor checks whether the model is still behaving. It can recommend a retrain. It cannot promote one — a person has to approve that, and a second person has to release it.

19:30

The bank file lands

Fresh balances, days overdue, and bank actions such as write-off or settlement. This is the day's new information.

19:45

The older scorecards refresh

Four hand-built scorecards update — repayment likelihood, recovery potential, visit priority, agent competency. None of them is this model, and none of them is trained from data. They are expert rules with weights.

20:00

The model runs, and the day is allocated

Each case's history is rebuilt exactly as it stood tonight, the model scores it, and the chance of recovery becomes the expected-rupees term in the engine that pairs cases to agents. Every score is written to a log table alongside the exact inputs that produced it.

06:00

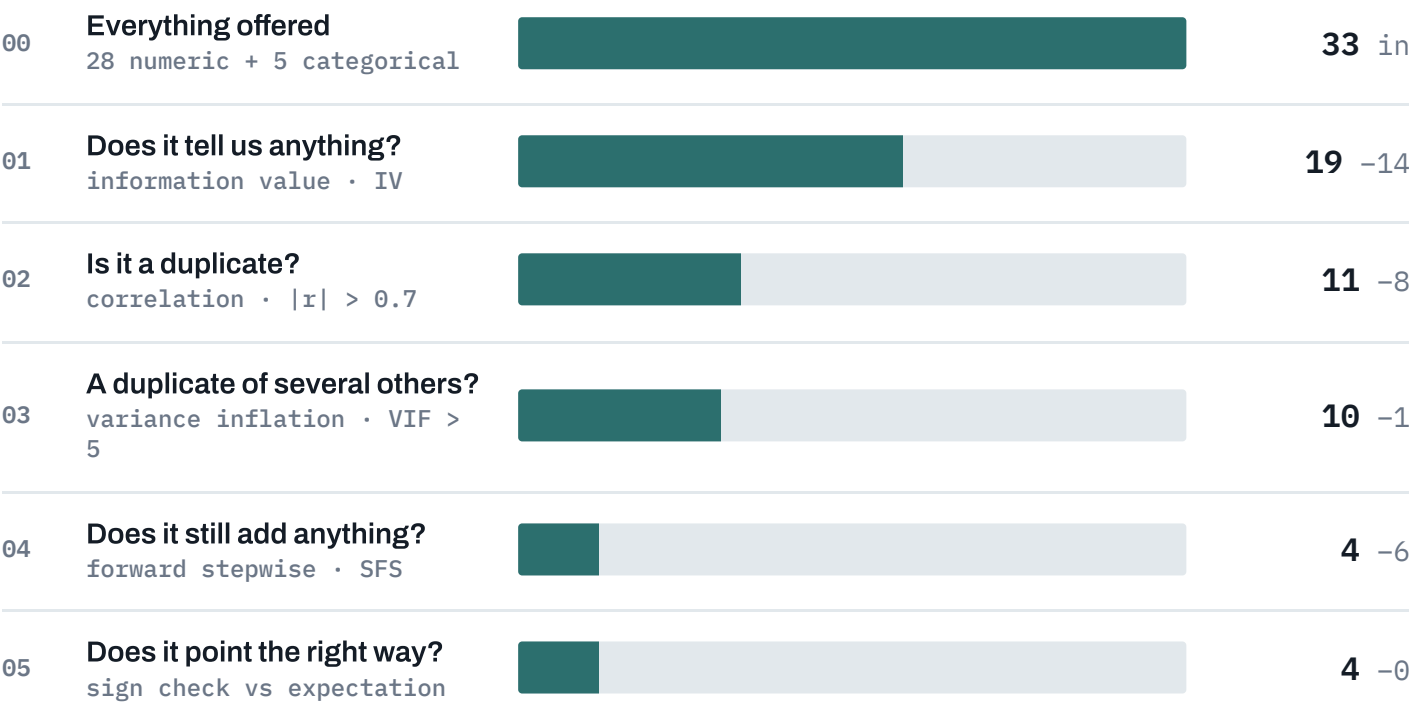
Agents get their day

Routed beats reach the phones. The manager screen shows the same figures with an ML-assisted badge and a plain-words tooltip on every case the model touched.

FEATURES — FROM 33 DOWN TO 4

Thirty-three things were offered to the model. Four survived, and every elimination round had to justify itself.

The starting pool was 33 candidate features — 28 numeric (days overdue, bureau score, arrears, payment ratios, visit and contact counts, promise history, income) and 5 categorical (loan type, overdue bucket, city, employment type, branch). Each round below removes features for one specific, stated reason. It is a funnel, so the order matters.



Round 2 is the one worth understanding. Days overdue and arrears ratio agree with each other 93% of the time, because they are two readings of the same fact. Keeping both would let one idea vote twice. Round 4 stopped adding features when the fifth candidate improved accuracy by 0.0002 — real, and far too small to be worth another thing that can break.

The four that made it, and how much each one counts

Bar length is that feature's maximum possible influence on the score. No single input can decide a case on its own.

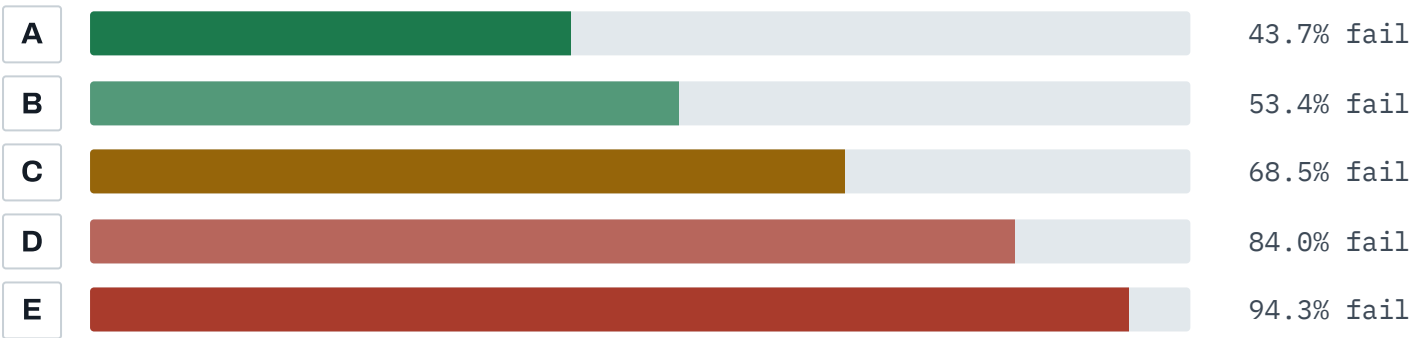


All four are things a collections manager would have named unprompted, which is the point: a model that cannot be explained to the person acting on it will not be trusted by them. This one is a **points system**, not a black box — each feature earns points, the points add up, and the total converts to a percentage. It fits on one page and can be argued with.

DOES THE RANKING ACTUALLY SEPARATE PEOPLE?

Sorted into five grades, the failure rate climbs from 44% to 94% without one step backwards.

This is the plainest evidence in the whole audit and it needs no statistics to read. Take 70,000 accounts, sort them by score, cut them into five grades. If the model works, each grade should be worse than the one above it.



Every step goes the right way — **zero rank-order breaks**. An A-grade borrower is more than twice as likely to pay as an E-grade one. That gap is what an agent's day is being spent on, and every metric in the next section measures it from a different angle.

EVERY METRIC, IN PLAIN WORDS

What each test asks, what counts as a good answer, and what this model scored.

Two rules apply throughout. Every number below is measured on 30,000 accounts from months the model never saw while training — the equivalent of marking an exam with questions the student was not shown. And several tests are *two-sided*: a score that is too good fails as hard as one that is too weak, because on a book like this, near-perfect accuracy almost always means the answer leaked into the question.

Metric	What it asks, in plain words	Good range	Ours	Verdict
Accuracy of ranking ROC-AUC	Pick one borrower who paid and one who did not. How often does the model score those two in the right order?	0.50 coin toss 0.70–0.80 useful >0.90 suspect	0.7568	PASS
Gini $2 \times \text{AUC} - 1$	The same thing rescaled, which is what the credit industry quotes. Zero is random guessing, one is perfect.	≥0.25 floor 0.35–0.55 target ≥0.60 suspect	0.5136 ±0.011	PASS
Separation KS statistic	At the single best cut-off, how wide is the gap between the payers and the non-payers?	≥20 required 30–45 strong ≥60 suspect	39.72	PASS
Predictive strength information value · IV	Per feature, before any modelling: how much does knowing this one thing change the odds? Used to throw out the useless and to flag the suspiciously good.	<0.02 useless 0.1–0.3 medium 0.3–0.5 strong >0.5 investigate	0.75 overdue days 0.34 overdue amt 0.28 promises 0.13 bureau	PASS
Re-expression WOE	Not a score — a translation. Each feature is cut into bands, and each band is replaced by "how much riskier than average is this band". It is what makes the model readable and stops one freak value dominating.	no range; bands must move in one direction	monotonic on all four	PASS
Redundancy VIF	Is this feature just a mix of the others wearing a different name? If so it adds noise and scrambles the weights.	<5 fine 5–10 drop it >10 severe	max 2.44 one dropped at 9.30	PASS
Selection SFS	Add features one at a time, best first, and stop when the next one stops earning its place. Prevents a bloated model that memorises instead of generalising.	stop when the gain < 0.0008	stopped at 4 5th gained 0.0002	PASS
Population drift PSI	Have the borrowers themselves changed since training? A model built on one kind of book quietly stops applying to a different one.	<0.10 stable 0.10–0.25 watch >0.25 retrain	0.0075	PASS
Input drift CSI	The same test applied to each input rather than to the final score — so you can see <i>which</i> input moved, not merely that something did. Same arithmetic, same thresholds.	<0.10 stable >0.25 retrain	max 0.0111 across the four used	PASS
Honesty of the % calibration gap	When it says 30%, do about 30 in 100 actually pay? Ranking correctly is not enough here, because this percentage gets multiplied by rupees.	within ±0.05 of observed	−0.0097	PASS

METRIC	WHAT IT ASKS, IN PLAIN WORDS	GOOD RANGE	OURS	VERDICT
Overall error Brier score	Average squared miss across every prediction. Rewards being right <i>and</i> being appropriately unsure.	0 perfect 0.25 no skill lower is better	0.1699	PASS
Order holds rank-order breaks	Going down the grades, does any grade have a <i>lower</i> failure rate than the one above it? Even one break means the ranking cannot be relied on.	0 breaks in the top 5	0	PASS
Memorising check train vs out-of-time	Is it much better on data it has seen than on data it has not? A big drop means it memorised rather than learned.	gap < 0.10	-0.0279 unseen scored higher	PASS

That last row is the most reassuring line in the table. The model performs *better* on the months it was never shown than on the months it trained on. A model that has memorised its training data cannot do that.

RUNNING IN PRODUCTION

Live counts, read from the database this morning.

19,935 scores recorded, each with the exact inputs behind it	973 distinct loans scored	100% of scores had every input available — none was guessed
214 of last night's allocations used the model	911 decisions traceable to the exact score behind them	0 predictions old enough to be marked yet

Every score carries an audit trail: model name, version, a fingerprint of the model file, the feature values used, the date they were read as of, and the decision they fed. When a manager asks "why was this case chosen", the answer is looked up rather than reconstructed.

If the model fails for any reason, the allocator falls back to its previous behaviour and the night's plan still gets made. That was learned the hard way: a date stored as text once broke the feature builder, and the system ran without the model for a full cycle while every health screen reported fine. It now logs that failure loudly. Failing safely and failing silently are not the same thing.

THE SCORES WE TESTED AND TURNED DOWN

The business already has two hand-built scores. We measured whether the model should read them, and the answer changed our reasoning as well as our answer.

The platform computes a **repayment score** (will this borrower pay?) and a **recovery score** (what share comes back, and how fast?). Both are older than the model and both are barred from being fed into it. In September 2026 we checked whether that bar was actually earned, or merely inherited.

First finding: the reason written in the code was wrong

The rule was justified on the grounds that these scores are "downstream outputs" and would therefore *leak* — leak meaning a model accidentally shown the answer, which then looks brilliant in testing and fails in the field. We traced the dates. The two windows never touch:

the scores look **BACKWARD** – from 180 days before the scoring day, up to that day
the model's target looks **FORWARD** – from that day until 30 days later

They are back to back and never overlap, and the code goes further: a promise whose status changed after the scoring day is deliberately thrown away, so yesterday's score cannot be quietly rewritten by today's news. **There is no leakage here.** The rule is sound; its stated reason was not, and that has been corrected wherever it was written down.

Second finding: the scores add nothing measurable

We rebuilt the champion from scratch and then rebuilt it five more times, each with a score forced in alongside the four features. Same data, same split, same maths, same tests. The rebuild reproduced the live model exactly, which is what makes the five comparisons worth reading.

WHAT THE MODEL WAS GIVEN	OUT-OF-TIME GINI	CHANGE	OUTCOME
The four features the live champion	0.5136	–	CHAMPION
+ repayment score repayment_likelihood	0.5131	–0.0005	REJECTED
+ risk score risk_score	0.5131	–0.0005	REJECTED

WHAT THE MODEL WAS GIVEN	OUT-OF-TIME GINI	CHANGE	OUTCOME
+ recovery rate recovery_rate_90	0.5135	-0.0001	REJECTED
+ expected recoverable expected_recoverable_amount	0.5134	-0.0002	REJECTED
+ all three at once	0.5130	-0.0006	REJECTED

Not one arm improved on the champion, and every one was turned down by the same gate the system uses to decide whether a retrained model may replace the live one. The rows are not rounding noise in the model's favour — they are all on the losing side of zero.

The risk score and the repayment score are the same number. One is literally `100 - the other`, and the two arms above produced bit-for-bit identical predictions. They are one input wearing two names, which is exactly why both stay on the barred list: so nobody adds "the other one" believing it is fresh evidence.

Third finding: the real reasons, which are better than the old one

- **Redundancy.** The four features already reconstruct the repayment score with 84% accuracy, and the score on its own separates payers from non-payers *less* well than days-overdue alone (0.432 against 0.468).
- **Double counting.** Adding the score drags the weight on days-overdue down from 0.797 to 0.664. It is not contributing new information; it is splitting the credit for information the model already had.
- **Version dependency.** These scores are hand-weighted and get re-tuned from time to time. Wire one in and a tuning session would shift the model's inputs without a single borrower having changed — setting off drift alarms that mean nothing.

What is still genuinely open

The bar applies to the two *scores*, never to the facts underneath them. Eight of those facts are things the model has never been offered: whether the borrower is **hostile**, a **fraud flag**, **legal action** under way, a **settlement** agreed, **difficult visit outcomes**, the **size of the last payment**, the **case target**, and **money paid in the recent window**.

Whether any of them would help is **unknown, and was not settled by this work**. They could not be tested: the simulated book the model trains on does not generate them. A possible future experiment — *recorded here as an idea, not scheduled and not built* — would be to start writing those raw facts into the prediction log alongside the four the model uses. That changes no score and no decision; it would simply mean that in thirty days' time the question could be answered with evidence instead of argument.

Nothing was changed by this investigation. The model, its four features, the barred list, the two scorecards and every production behaviour are exactly as they were. The only thing that moved is the written reasoning, which now matches what the data says.

WHAT THIS AUDIT DOES NOT PROVE

Four limits, stated plainly, because each is the sort of thing that gets quoted out of context later.

Read these alongside every number above

- **It was trained on simulated borrowers, not real ones.** The 120,000 training rows come from a synthetic book. Everything here proves the process works end to end and reports believable results. None of it is evidence about how much money this recovers from real people.
- **No prediction has been marked right or wrong yet.** The first ones reach their 30-day maturity on 2026-10-08, and the monitor needs 500 matured accounts before it will report live performance at all — below that the verdict would be noise wearing a number. Today it correctly says "not ready".
- **The training months and the test months are visibly different books.** A check designed to detect exactly that scored 0.75, where 0.50 would mean identical. So part of the out-of-time result measures how well the model survives a changing book rather than pure skill. Worth stating, and it cuts both ways: it held up across that change.
- **A second model was built and rejected.** `contact_risk`, which tried to predict whether a visit would find the borrower at home, reached a Gini of 0.19 against a 0.25 floor. It failed its gates, was never promoted, and is used nowhere. It is kept so the failure can be read. The honest diagnosis is that the data it needs — time of day, phone and address quality — is not captured by the product yet, which makes it a data problem rather than a modelling one.

Figures read directly from the model artifact at `ml/artifacts/recovery_risk/1.1.0` and from the live `model_predictions` and `allocation_decisions` tables on 2026-09-11. Nothing on this page is estimated or recalled from documentation. The six ablation arms were rebuilt on the 120,000-row modelling panel through the production pipeline modules — same split, binning, calibration and evaluation — and judged by `comparison.compare_to_incumbent`, the same gate that decides whether a retrained model may go live. The rebuilt champion reproduced the committed artifact exactly, which is what makes the five comparisons against it meaningful.